

Apprentissage Profond (I-ILIA-029) -- Travaux pratiques

TP 07 : Explicabilité CNN & ViT pour la classification d'objets dans un chantier ferroviaire

Lors des TP5 et TP6, des réseaux neuronaux profonds poussés ont été utilisés pour résoudre de la classification d'objets dans un chantier ferroviaire (CNNs, ViTs). Un problème avéré par l'état de l'art est que l'utilisation des réseaux neuronaux de plus en plus profonds (paramètres +++) rend l'interprétabilité des résultats complexes ; on appelle leur comportement souvent de « **boîte noire** ». Le réseau nous produit une décision avec de bonnes performances, mais on ne comprend pas exactement sur quels éléments le réseau appuie sa décision. Ainsi, l'objectif de ce TP est d'obtenir une explication des résultats produits par les réseaux neuronaux précédemment utilisés lors du TP5 et TP6 (CNN – ViT). Le problème initial reste donc inchangé : développer un module de vidéo surveillance qui consiste à classer des images représentant les objets (rails de train, train, pelleteuse) et ouvriers dans un chantier ferroviaire. Des techniques d'explicabilité seront utilisées dans un premier temps pour comprendre les résultats des réseaux CNNs ; et des techniques adaptées aux ViTs seront ensuite testées pour comprendre leurs résultats. Enfin, nous vous proposons de manière optionnelle d'étudier plus en profondeur le fonctionnement d'une méthode d'explicabilité basée sur la perturbation : « RISE ».



Figure 1 : Exemple d'images représentant les classes à identifier

Enoncé du TP :

- **Question 1** : importer le notebook « *TP7_XAI_ILIA_029_Input_2025.ipynb* » et vérifier que le GPU est bien sélectionné ;
- **Instruction 1 « Installation »** : Installer la librairie Pytorch Lightning ;
- **Question 2** : connecter votre notebook à la plateforme **WanDB** ;
- **Question 3** : télécharger le jeu de données d'entraînement : **railway-construction-100**
 - BD railway-construction-100: <https://nextcloud.ig.umons.ac.be/s/TaNbLXW8BrES5Te/download/railway-construction-100.zip>
- **Instruction 2 : « Importation des libraires »** : importer les libraires nécessaires ;

- **Partie 1 : Entraînement de modèles CNN :**

- **Instruction 3 :** définir les paramètres généraux d'entraînement;
- **Question 4 :** créer la classe «Model» vue au TP5 pour appliquer un transfert d'apprentissage à partir du modèle «**efficientnet_v2_s**», la fonction d'erreur: **CrossEntropyLoss**, l'optimiseur **SGD**, avec des poids de modèle définis en tant que « **DEFAULT** » dans le constructeur de la classe, et avec avec l'ordre des classes sauvé sous **self.classes**.

Notez la présence de «**weights**» et «**classes**» en tant que paramètres initiaux de la classe pour la suite du TP, à ne pas supprimer !

- **Question 5 :** créer la fonction de division des données et création des dataloaders « **create_data_loaders** ».
- **Question 6 :** créer les data loaders (train, val, test) avec la fonction **create_data_loaders**
- **Instruction 5 :** définir une variable "class_names" comme étant l'ordre des classes utilisées par le jeu de données, obtainable à partir de n'importe quel loader (train, val, test) défini en question 4.
- **Question 7 :** lancer l'entraînement :
 - Modèle EfficientNet avec l'ordre des classes donnée en paramètre initial.
 - Pas d'apprentissage : '0.01'
- **Question 7.1 :** définir la fonction de visualisation des courbes d'entraînement avec plot
- **Question 7.1 :** afficher vos courbes d'entraînement avec plot et/ou WandDB
- **Question 8 :** évaluer votre modèle avec les données de test (avec la matrice de confusion)
- **Question 9 :** calculer les métriques de précision, de rappel, et de score F1 sur base de la matrice de confusion et note dans ton code spécifiquement quelle classe présente le/la meilleur(e) et inversement le/la pire :
 - Précision ?
 - Rappel ?
 - Score-F1 ?
- **Question 10 :** proposer 3 façons (sans les appliquer) pour essayer d'améliorer la performance du modèle,

liée ou non aux observations tirées sur les classes via la matrice de confusion.

- **Partie 2 : Explicabilité CNN**

Cette première partie liée à l'explicabilité vous propose d'utiliser les librairies **torchcam** et **captum** pour analyser les résultats obtenus sur des images de différentes classes sur votre modèle entraîné, ainsi qu'un autre modèle non entraîné, avec des méthodes disponibles des librairies.

- **Instruction 6** : installer les librairies torchcam et captum
- **Instructions 7** : importer les fonctions/classes nécessaires à la bonne exécution des méthodes d'explicabilité
- **Question 10** : créer un modèle EfficientNet nommé « **rand_model** », qui sera **NON entraîné et SANS poids prédéfinis** (poids aléatoire !) afin de comparer l'explicabilité entre un modèle entraîné et un modèle non entraîné
- **Instruction 8** : analyser exécuter la fonction d'extraction d'images de test par classe
- **Instruction 9** : analyser et exécute la fonction d'explication contenant un ensemble de méthodes d'explicabilité pour CNN
- **Instruction 10** : choisir un nombre d'images par classe à extraire, une classe, une méthode XAI et le modèle entraîné pour obtenir des résultats sur les images spécifiées ci-dessous
- **Question 11** : extraire les images par classe qui seront analysées
- **Question 12** : compléter le code pour expliquer les résultats du modèle sur les images extraites.
- **Question 13** : faites tourner le code avec les différentes méthodes sur le modèle entraîné. Quelle méthode vous semble donner de meilleurs résultats ?
- **Question 14** : analyser les résultats en faisant tourner le code pour chacune des classes sur cette méthode XAI. Observez-les bonnes et mauvaises prédictions et proposez une manière d'améliorer le modèle.
- **Question 15** : Utiliser maintenant le modèle non entraîné. Quelles différences constatez-vous entre le modèle entraîné et le modèle sans poids prédéfinis ?

- **Partie 3 – Entraînement ViT et explicabilité**

- **Question 16 :** Définir un modèle ViT - similaire au TP6 - en gardant le paramètre initial de 'classes' existant utilisé pour la matrice de confusion du modèle CNN.
- **Question 17 :** Entraîner le modèle ViT : fonction d'erreur: **CrossEntropyLoss**, optimiseur **SGD**, et avec des poids de modèle définis en tant que « **DEFAULT** » dans le constructeur de la classe.
- **Question 18:** Utiliser la matrice de confusion pour tester le modèle ViT.
- **Instruction 11 :** Télécharger le Github « Transformer Input Sampling » et installer les imports nécessaires à son bon fonctionnement.
- **Instruction 12 :** Exécuter la fonction d'explication contenant un ensemble de méthodes d'explicabilité pour ViT
- **Question 19 :** Compléter le code en sélectionnant une méthode d'explicabilité initiale et le modèle ViT, et appliquer la fonction "explain_model_vit".
- **Question 20 :** Faites tourner le code avec les différentes méthodes sur le modèle ViT. Quelle méthode vous semble donner de meilleurs résultats ?
- **Question 21 :** Analyser maintenant les résultats en faisant tourner le code pour chacune des classes sur cette méthode XAI. Observez-les bonnes et mauvaises prédictions et proposez une manière d'améliorer le modèle.

- **(Optionnel) Partie 4 – Explicabilité via la méthode de perturbation « RISE ».**

Le principe fondamental des méthodes de perturbation visant à générer une explication du résultat d'un modèle Deep Learning est de perturber l'image d'entrée originale (= supprimer des pixels, des zones de pixels de l'image initiale) et de calculer le score obtenu sur cette image perturbée.

En répétant cette opération des centaines/ milliers de fois selon la taille de l'image, on peut trouver quelles parties de l'image sont les plus importantes pour la décision d'un réseau. Une méthode fonctionnant bien dans l'état de l'art du milieu XAI est la méthode « RISE » dont voici le schéma de fonctionnement ci-dessous :

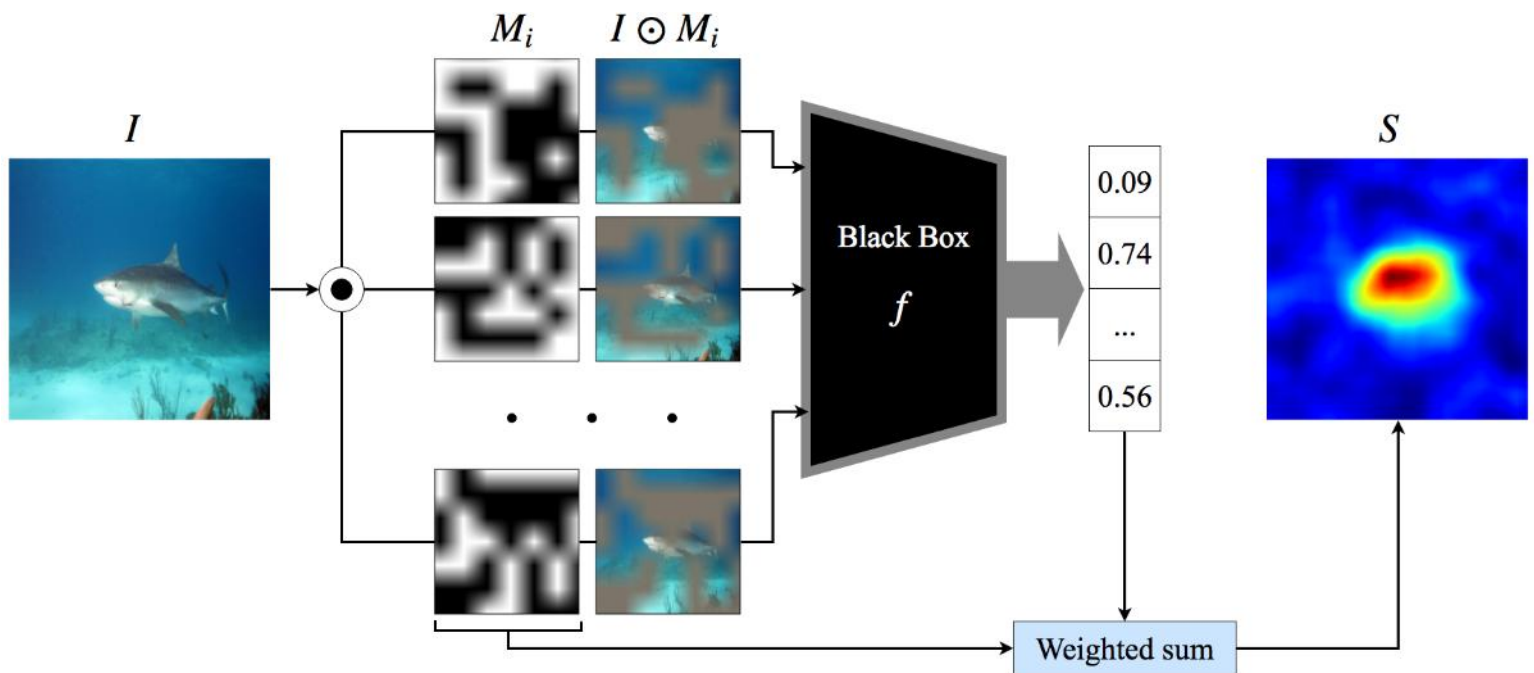


Figure 1: Schéma de la méthode RISE [1]

[1] Petsiuk, V., Das, A., & Saenko, K. (2018). Rise: Randomized input sampling for explanation of black-box models. arXiv preprint arXiv:1806.07421.

Il vous est demandé de **compléter le code** suivant (inspiré de [1]) pour aboutir au schéma de la Figure 1, dont voici les étapes principales (détaillées dans le TP si nécessaire) :

1. Créer des masques aléatoires de pixels à appliquer sur une image ;
2. Application de ceux-ci sur l'image.
3. Prédiction des images avec masques par le modèle AI.
4. Combinaison linéaire des masques par un poids équivalent à la prédiction du point 3) pour produire la carte (heatmap) d'explicabilité.